

C# - Phần 10.5: Count(), Sum(), Average(), Min(), Max()

Đánh giá bài làm - Đáp án - Giải thích - Lưu ý

Kết quả: Bài làm đạt khoảng 9.3/10 - ĐẠT. Phần code gần như đúng toàn bộ; điểm cần chỉnh chủ yếu là naming, exception cụ thể và cách diễn đạt câu lý thuyết về Min().

PHẦN A - ĐÁNH GIÁ CODE

1. Count() - đúng

```
int productCount = products.Count();

int productCount2 =
    products.Count(product => product.Price >= 1000);
```

Count() đếm toàn bộ phần tử. Count(condition) đếm số phần tử thỏa điều kiện Lambda. Với dữ liệu bài tập: tổng số Product = 5, số Product có Price >= 1000 = 2.

2. Sum() - đúng

```
decimal totalPrice =
    products.Sum(product => product.Price);
```

Kết quả là 18650. Lambda có dạng Product -> decimal; Sum() cộng toàn bộ các giá trị decimal mà Lambda lấy ra.

```
decimal totalPriceUnder1000 =
    products
        .Where(product => product.Price < 1000)
        .Sum(product => product.Price);
```

Kết quả là 1150. Where() lọc Product, sau đó Sum() cộng Price của các Product còn lại.

3. Average() - đúng

```
decimal averagePrice =
    products.Average(product => product.Price);

decimal averagePriceOver1000 =
    products
        .Where(product => product.Price >= 1000)
        .Average(product => product.Price);
```

Average Price toàn bộ = 3730. Average của nhóm Price >= 1000 = 8750.

4. Min() và Max() - đúng

```
decimal minPrice =
    products.Min(product => product.Price);

decimal maxPrice =
    products.Max(product => product.Price);
```

minPrice = 150; maxPrice = 15000. Cả hai method trên trả giá trị decimal, không tự trả về Product.

5. Tìm Product từ minPrice / maxPrice - đúng

```
Product? cheapestProduct =
    products.FirstOrDefault(
        product => product.Price == minPrice);

Product? mostExpensiveProduct =
    products.FirstOrDefault(
        product => product.Price == maxPrice);
```

Kết quả tương ứng là USB và Laptop. Nếu có nhiều Product cùng mức giá nhỏ nhất hoặc lớn nhất, FirstOrDefault() chỉ lấy phần tử đầu tiên thỏa điều kiện.

6. Collection rỗng - xử lý đúng

```
List<Product> emptyProduct = new List<Product>();

int count = emptyProduct.Count();           // 0
decimal sum = emptyProduct.Sum(
    p => p.Price);                           // 0
```

Count() trên collection rỗng trả 0. Sum() với kiểu số trong trường hợp này cũng trả 0.

```
try
{
    decimal avg =
        emptyProduct.Average(p => p.Price);
}
catch (InvalidOperationException e)
{
    Console.WriteLine(e.Message);
}
```

Average() trên collection rỗng throw InvalidOperationException với thông báo kiểu "Sequence contains no elements".

PHẦN B - CÁC ĐIỂM CẦN CHỈNH TRONG CODE

- Tên biến productCount3 và productCount4 chưa đúng nghĩa vì đó là tổng giá, không phải số lượng. Nên dùng totalPrice, totalPriceUnder1000, averagePrice...
- Các dòng Console.WriteLine dùng nhãn "quantity" cho Sum/Average là chưa phù hợp; nên ghi Total price hoặc Average price.
- Khi đã biết lỗi có thể xảy ra là InvalidOperationException, nên catch loại cụ thể thay vì catch(Exception).
- using System.Data và using System.Net.Http.Headers không được dùng trong bài này nên có thể bỏ.

PHẦN C - ĐÁNH GIÁ CÂU HỎI LÝ THUYẾT

1. Count() dùng để làm gì?

Đúng: Count() đếm số lượng phần tử trong collection; Count(condition) đếm số phần tử thỏa Lambda.

2. Count() và Any() khác nhau thế nào?

Đúng: Count() trả int và trả lời "bao nhiêu?"; Any() trả bool và trả lời "có hay không?".

3. Sum(product => product.Price) có nghĩa gì?

Đúng: với mỗi Product lấy Price kiểu decimal rồi cộng tất cả Price lại.

4. Average() dùng để làm gì?

Đúng: dùng để tính trung bình cộng của các giá trị số được chọn.

5. Min(product => product.Price) trả gì?

Đúng: trả decimal, không trả Product.

6. Vì sao chỉ Min() chưa đủ để lấy Product rẻ nhất?

Cần chỉnh: lý do chính là Min(product => product.Price) chỉ trả giá trị decimal nhỏ nhất. Muốn lấy object Product tương ứng cần truy vấn tiếp, ví dụ FirstOrDefault(product => product.Price == minPrice).

7. Count() trên collection rỗng?

Đúng: trả 0.

8. Sum() trên collection rỗng?

Đúng trong trường hợp đang học: trả 0.

9. Average() trên collection rỗng?

Đúng: throw InvalidOperationException.

10. Where + Sum thay đổi dữ liệu thế nào?

Đúng: List<Product> -> IEnumerable<Product> sau Where -> decimal sau Sum.

PHẦN D - ĐÁP ÁN CODE THAM KHẢO

```
public class Program
{
    public static void Main(string[] args)
    {
        List<Product> products = new List<Product>
        {
            new Product("P01", "Mouse", 300),
            new Product("P02", "Keyboard", 700),
            new Product("P03", "Monitor", 2500),
            new Product("P04", "Laptop", 15000),
            new Product("P05", "USB", 150)
        };

        int productCount = products.Count();
        int expensiveCount =
            products.Count(p => p.Price >= 1000);

        decimal totalPrice =
            products.Sum(p => p.Price);

        decimal totalPriceUnder1000 =
            products
                .Where(p => p.Price < 1000)
                .Sum(p => p.Price);

        decimal averagePrice =
            products.Average(p => p.Price);

        decimal averagePriceOver1000 =
            products
                .Where(p => p.Price >= 1000)
                .Average(p => p.Price);

        decimal minPrice =
            products.Min(p => p.Price);

        decimal maxPrice =
            products.Max(p => p.Price);

        Product? cheapestProduct =
            products.FirstOrDefault(
                p => p.Price == minPrice);

        Product? mostExpensiveProduct =
            products.FirstOrDefault(
                p => p.Price == maxPrice);

        List<Product> emptyProducts =
            new List<Product>();

        Console.WriteLine(emptyProducts.Count());
        Console.WriteLine(
            emptyProducts.Sum(p => p.Price));

        try
        {
            decimal avg =
                emptyProducts.Average(p => p.Price);
        }
        catch (InvalidOperationException e)
        {
            //
        }
    }
}
```

```

        Console.WriteLine(e.Message);
    }
}

```

PHẦN E - LƯU Ý QUAN TRỌNG

Count()
→ int

Sum() / Average() / Min() / Max()
→ giá trị số

Min(selector) / Max(selector)
→ trả giá trị được chọn
→ không tự trả object

Average(empty)
→ InvalidOperationException

Phản xạ LINQ nên giữ: Mỗi khi đọc một chuỗi LINQ, hãy theo dõi cả mục đích lẫn kiểu dữ liệu sau từng bước. Ví dụ: List<Product> -> IEnumerable<Product> sau Where -> decimal sau Sum.

PHẦN F - TỔNG KẾT

Method	Mục đích	Kết quả
Count()	Đếm	int
Sum()	Tính tổng	Kiểu số
Average()	Tính trung bình	Kiểu số
Min()	Lấy giá trị nhỏ nhất	Kiểu số/kiểu được chọn
Max()	Lấy giá trị lớn nhất	Kiểu số/kiểu được chọn

Điểm bài: 9.3/10 - ĐẠT. Có thể chuyển sang 10.6: OrderBy(), OrderByDescending(), ThenBy(), ThenByDescending().